22/04/26

Name- Devansh Garg (2301350025)

BTECH CSE FSD

Capstone Assignment COA.

Submitted to Manikajal.

Case Study 4: Cache Design for an Online Examination System.

Introduction

In the modern era of digital education examination system have become a critical infrastructure component for universities, competitive examination bodies, and corporate training platforms these system are accessed simultaneously by thousands - sometimes ten of thousands - of students during peak examination hours. Every student expect near instantaneous page load, quick question, rendering and seamless answer submission. Any lag in response can cause frustration, impair time loss, and in serious cases, system crashes that compromise the integrity of the examination. The core challenge behind this performance issue is not just network bandwidth or server count - it fundamentally lies in how efficiently instructions and data are fetched from memory inside the server powering the platform. When thousands of concurrent requests hit the server the processor must repeatedly fetch intruction and data, banks, student records from memory. If every fetch has to go all the way to the main memory (RAM) or worse, to disk storage the system will be unacceptable

hierarchy and specifically cache memory becomes the most powerful architectural tool. A well designed cache strategy dramatically reduces the time taken to fetch frequently used data, enabling the system to handle massive concurrent load without degrading performance.

Concept from COA

(1) Locality of Reference

The principle of locality of reference is the foundational reason why cache work so effectively. It states upon perform program and data accesses tend to be clustered both in time and space.

Temporaly locality

means that if a particular memory location is accessed it is very likely to be accessed again in the near future. In an online exam system. the question paper data timing logic.

Proposed Analysis And Design

Cache Mapping Techniques

One of the most important design decision in cache architecture is determining how blocks of

main memory are mapped to location in the cache. Since the cache is far smaller than main memory, many memory blocks must share the same cache ~~memory~~ space. The mapping policy decides the rule for this sharing, and the choice dramatically affect how often the cache can satisfy a request without going back to main memory. There are three fundamental approaches: direct mapping, fully associative mapping and set associative mapping.

Direct mapping

Direct mapping is the simplest and most straightforward cache mapping strategy in this approach every block in main memory has exactly one specific location in the cache where it can be stored. The cache line to which ~~a~~ memory block is assigned is determined by a simple modulo operation. If the cache has say 128 lines, then memory block number 0 map to line cache 0.

To check whether a requested memory is in the cache the hardware computes the cache line index from the block address goes directly to that line and compares the stored tag with the tag of the requested address. If they match it is a cache hit the data ~~and the~~ is returned instantly. If they do not ~~catch~~ match it is a cache miss and block must be fetched from main memory replacing whatever was

previously sitting in the cache line. The great advantage of direct mapping is its simplicity and speed. The hardware is cheap to build and the lookup is almost instantaneous because there is only ever one place to look. However direct mapping carries a serious weakness known as conflict misses. Consider a scenario in the online exam system where the question rendering code and the student session management code both happen to map the same cache line. Every time the processes switch between executing these two modules - which happen constantly during exam processes it evicts on block to ~~look~~ load the other. Even if every other cache line is completely empty these two blocks keep throwing each other out. This phenomenon is called cache thrashing and it can severely degrade performance in real world application. with multiple active data streams, which is exactly the situation is high concurrency exam server.

(B) Fully Associative Mapping

Fully associative mapping takes the opposite extreme approach. In this strategy a memory block can be placed in absolutely any cache line there are no restriction whatsoever. When the processor needs to find a block in the cache. It must search every single cache line simultaneously

to see if any of them holds the requiring block. This parallel search is performed by comparing the tag of the requested address against the tags of all cache lines at the same time, using dedicated comparison hardware called comparators. The benefit of this approach is maximum flexibility. Since any block can go anywhere conflict misses are entirely eliminated. The cache is used with maximum efficiency - a line is only displaced when the cache is genuinely full and there is no empty space not because of arbitrary mapping restrictions. For an online exam system this would mean the question data, session data, authentication token and application code could all exist in cache without evicting each other arbitrarilly.

However the hardware cost is the fatal drawback. For every cache line a separate comparator circuit must be built and all comparison must happen in parallel to keep lookup time fast. ~~At each~~ As cache size grows into the megabyte range, as required for a server class system, the number of comparator required becomes enormous making the design prohibitively expensive power hungry and physically large. For this reason fully associative

mapping.

Set Associative Mapping

Set associative mapping is the design philosphy used in virtually every modern high performance processor and for good reason. It elegantly combines the strength of both direct mapping and fully associative mapping while avoiding the worst weakness of each.

In set associative mapping the cache is divided into a number of groups called sets. Each sets contained a fixed number of cache lines called ways. A memory block is restricted to mapping to single specific set just like in direct mapping the set is determined by modulo operation one the block address. However within that set the block can be placed in any of the available ways. just like in fully associative mapping within a small scope. For example in a 4way set associative cache the cache is divided into many sets and each set holds exactly 4 lines. A memory block always map to one same set index no longer evict each other, they wait.

For online system recommendation system 8-way set associative L3 cache is the ideal recommendation. The concurrent access pattern generated by thousand of students

Simultaneously requesting different quiting submitting answers and authenticating sessions create a highly varied mix of memory access. A 8 way associate cache can accomodate this variety ~~of~~ with few conflict misses, ensuring consistently high cache hit rates.

Conclusion

This case study has demonstrated with both conceptual and quantitative evidence that the performance of an online examination system is fundamentally governed by the principles of cache memory design in computer organization and architecture. The challenge of serving thousand of simultaneous user without degradation is not solved by simple buying more hardware it is solved by intelligently designing the memory hierarchy so that the processor spend as little time as possible waiting for data.